МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №2
по курсу «ПГП и ТПП»

Обработка изображений на GPU. Фильтры

Выполнил: Кудрявов Л.В.
Группа: М8О-406Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

1. **Цель работы.** Научиться использовать GPU для обработки изображений: текстурная память и двухмерная сетка нитей. Изображение хранится в бинарном файле: два целых числа (ширина w и высота h, по 4 байта), затем по построчно w·h пикселей, каждый из которых — четыре байта (r, g, b, a). Используются только цветовые компоненты, альфа-канал не учитывается; допускается ошибка в ±1. Ограничения: $w < 2^{16}$, $h < 2^{16}$.
2. **Вариант 7. Выделение контуров, метод Собеля.** Входные данные: путь к исходному изображению и путь к результату (по строке). Выходной файл содержит изображение того же формата. Пиксели за границей изображения получаются дублированием граничных пикселей.

Программное и аппаратное обеспечение

Вычисления выполнялись в облачной среде Google Colab с графическим ускорителем NVIDIA. Характеристики графического процессора приведены в таблице 1. Значения даны для NVIDIA Tesla T4 (compute capability 7.5, та же архитектура, что у GeForce GTX 1650 проверяющей системы). Характеристики соответствуют стандартной конфигурации Google Colab с ускорителем T4.

Таблица 1 — Характеристики оборудования

Параметр	Значение
Графический процессор	NVIDIA Tesla T4
Compute capability	7.5
Графическая (глобальная) память	15 360 МиБ (≈ 15 ГБ), GDDR6
Разделяемая память на блок	48 КБ (64 КБ на мультипроцессор)
Константная память	64 КБ
Регистров на блок	65 536
Максимум нитей в блоке	1024 (размеры блока до 1024 × 1024 × 64)
Максимальный размер сетки	$2^{31} - 1 \times 65\,535 \times 65\,535$ блоков
Число мультипроцессоров	40
Размер варпа	32 нити
Центральный процессор	Intel(R) Xeon(R) CPU @ 2.00GHz, 2 логических ядра
Оперативная память	≈ 12 ГБ
Дисковая память	≈ 100 ГБ

Проверяющая система курса: GeForce GTX 1650 (compute capability 7.5, 4 ГБ глобальной памяти, 48 КБ разделяемой памяти на блок, 65 536 регистров на блок), nvcc 12.0, g++ 13.3.0, Ubuntu 24.04.1 LTS.

Программное обеспечение: операционная система Ubuntu 24.04 (среда выполнения Google Colab), компилятор nvcc из состава CUDA Toolkit 13.0 (V13.0.88), компилятор g++ 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1), Python 3 с библиотеками NumPy, pandas, matplotlib и Pillow (генерация тестов, конвертер изображений,

независимая проверка результатов, таблицы). Среда разработки: ноутбуки Jupyter в Google Colab, текстовый редактор. Сдаваемые программы собираются командой `nvcc -arch=sm_75 -Werror cross-execution-space-call -lm`, профилирующие — `nvcc -O2 -arch=sm_75`. Версия nvcc проверяющей системы (12.0) отличается от использованной, но программы применяют только базовые возможности CUDA.

Метод решения

Для каждого пикселя вычисляется яркость по формуле Y = 0,299·R + 0,587·G + 0,114·B. Затем по окну 3 × 3 вокруг пикселя считаются приближения горизонтальной и вертикальной производных яркости масками Собеля:

```
Gx = (Y13 + 2*Y23 + Y33) - (Y11 + 2*Y21 + Y31)
Gy = (Y31 + 2*Y32 + Y33) - (Y11 + 2*Y12 + Y13)
G  = min(255, sqrt(Gx^2 + Gy^2))
```

Здесь Y_{ij} — яркость пикселя окна (i — строка, j — столбец). Выходной пиксель имеет r = g = b = G, значение альфа-канала сохраняется из исходного пикселя. Большое значение G соответствует резкому перепаду яркости, то есть контуру.

Исходное изображение размещается в текстурной памяти (`cudaArray` и текстурный объект `cudaTextureObject_t`). Это даёт два преимущества: кэш текстур оптимизирован под двумерную локальность доступа (окно 3 × 3 соседних пикселей), а режим адресации `cudaAddressModeClamp` аппаратно дублирует граничные пиксели и избавляет от проверок границ в ядре. Фильтрация отключена (`cudaFilterModePoint`), координаты заданы в пикселях, обращение идёт в центр текселя (+0,5).

Нить обрабатывает один пиксель; используется двумерная сетка нитей: по оси x — столбцы, по оси y — строки, поэтому нити одного варпа обращаются к соседним пикселям строки. Размер сетки фиксирован (`<<<dim3(16, 16), dim3(32, 32)>>>`, 262 144 нити) и не зависит от размера изображения: пиксели обходятся вложенными циклами с шагом, равным размеру сетки по каждой оси. Вычислительная сложность O(w·h), восемь текстурных выборок на пиксель.

Использованные источники: Гонсалес Р., Вудс Р. Цифровая обработка изображений (М.: Техносфера); NVIDIA CUDA C++ Programming Guide (раздел Texture Memory); лекции курса (фильтры обработки изображений).

Описание программы

Программа находится в файле `lab2.cu`; для работы с форматом изображений написан конвертер `converter.py` (PNG/JPG в бинарный формат и обратно, библиотеки NumPy и Pillow).

- `brightness(uchar4 p)` — функция устройства, яркость пикселя.
- `sobel(tex, out, w, h)` — ядро: чтение восьми соседей через `tex2D<uchar4>`, вычисление Gx, Gy и G, запись пикселя.

- `main` — чтение путей и изображения с проверками, создание `cudaArray` (`cudaMallocArray`, `cudaMemcpy2DToArray`), текстурного объекта (клампинг, точечная выборка, ненормированные координаты), запуск ядра, запись результата. Ошибки сопровождаются сообщением `ERROR: …` в stderr и завершением с кодом 0.

```
__global__ void sobel(cudaTextureObject_t tex, uchar4 *out, int w, int h) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int idy = blockIdx.y * blockDim.y + threadIdx.y;
    int offsetx = blockDim.x * gridDim.x, offsety = blockDim.y * gridDim.y;
    for (int y = idy; y < h; y += offsety)
        for (int x = idx; x < w; x += offsetx) {
            float fx = x + 0.5f, fy = y + 0.5f;
            float y11 = brightness(tex2D<uchar4>(tex, fx - 1.0f, fy - 1.0f));
            /* ... ещё семь выборок окна 3x3 ... */
            float gx = (y13 + 2.0f * y23 + y33) - (y11 + 2.0f * y21 + y31);
            float gy = (y31 + 2.0f * y32 + y33) - (y11 + 2.0f * y12 + y13);
            float g = fminf(255.0f, sqrtf(gx * gx + gy * gy));
            uchar4 src = tex2D<uchar4>(tex, fx, fy);
            out[y * w + x] = make_uchar4((unsigned char)g, (unsigned char)g,
(unsigned char)g, src.w);
        }
}
```

Результаты

1. Проверка корректности

Таблица 2 — Тесты программы

Тест	Ожидаемый результат	Результат
Пример 3 × 3 (пиксели 010203, 040506, 070809, 090807, 060504, 030201, 000000, 141414, 000000)	14 0C 11 13 15 18 39 14 3F (значение r = g = b по пикселям)	совпадает
Пример 3 × 3: один серый пиксель 80 в центре чёрного поля	B5 FF B5 FF 00 FF B5 FF B5	совпадает
Изображение 1 × 5 с серым пикселем 64 в центре	00 FF 00 FF 00	совпадает
Случайные изображения 1 × 1, 1 × 5, 7 × 1, 37 × 23, 300 × 200	совпадение с независимой реализацией на Python, отклонение 0	совпадает
Несуществующий входной файл	ERROR в stderr, код возврата 0	совпадает

Проверка на GPU (Tesla T4, Google Colab): скрипт `check_lab2.py` на тестовых файлах курса (`02.data`, `57.data`, `95.data`–`99.data`) показал максимальное отклонение 0 для всех файлов, у всех пикселей результата r = g = b. При замерах результат GPU сравнивался с CPU-реализацией на изображениях до 10000 × 10000: пикселей, различающихся более чем на 1, нет.

2. Скриншоты

На рисунке 1 показан результат работы программы на синтетическом изображении 480 × 300: слева исходное изображение, справа результат выделения контуров. Контуры фигур и надписи выделены белым цветом, области однородного

цвета — чёрным. В Google Colab изображения не отобразились из-за плохого разрешения. На рисунке 1 изображен пример.

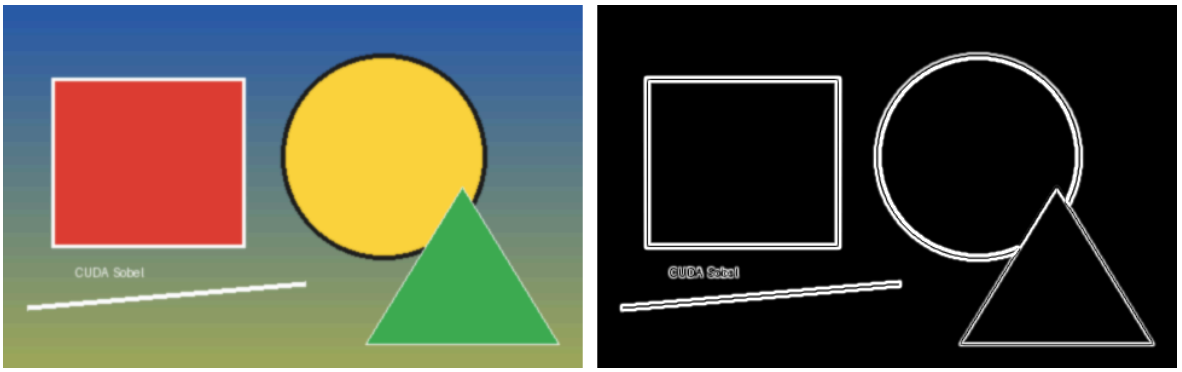


Рисунок 1 — Исходное изображение (слева) и результат метода Собеля (справа)

3. Время работы ядра при разных конфигурациях запуска

Замеры выполнены профилирующей программой `lab2_bench.cu`: ядро то же, что в `lab2.cu`, изображение (клетки с шумом) генерируется внутри программы, сборка `nvcc -O2 -arch=sm_75`. Время ядра измерялось событиями `cudaEvent` и усреднялось по 5 запускам после прогрева (если первый запуск длился больше 1 с, выполнялся один запуск). Конфигурация записана как `<<<(gx, gy), (bx, by)>>>`; сдаваемая версия использует `<<<(16, 16), (32, 32)>>>`.

Таблица 3 — Время работы ядра, мс

Конфигурация	640 × 480	4096 × 4096	10000 × 10000
<<<(1, 1), (32, 1)>>>	8,632	273,85	1 634,81
<<<(1, 1), (32, 32)>>>	1,076	32,79	158,57
<<<(4, 4), (16, 16)>>>	0,102	5,518	35,04
<<<(16, 16), (32, 32)>>>	0,0412	1,260	7,348
<<<(32, 32), (32, 32)>>>	0,0412	1,194	7,215

Для изображения 10000 × 10000 время ядра уменьшается с 1 634,81 мс (`<<<(1, 1), (32, 1)>>>`, один варп) до 7,348 мс для сдаваемой конфигурации, то есть в 222 раз. Один блок из 1024 нитей (158,57 мс) выполняется на одном мультипроцессоре из 40 и не загружает устройство; сетка 4 × 4 блоков (35,04 мс) использует только 16 мультипроцессоров. Начиная с 16 × 16 блоков по 1024 нити (262 144 нити) время не уменьшается: `<<<(32, 32), (32, 32)>>>` дало 7,215 мс. Лучшая конфигурация обрабатывает 13,9 млрд пикселей в секунду, что при чтении и записи по 4 байта на пиксель соответствует около 111 ГБ/с по уникальным данным (соседние пиксели окна читаются из кэша текстур). Значения таблиц 3 и 4 получены в разных запусках, поэтому отличаются в пределах разброса измерений.

4. Сравнение с CPU

Для сравнения в той же программе выполняется последовательная версия фильтра на CPU (один поток, Intel Xeon 2,00 ГГц) с тем же алгоритмом и тем же правилом обработки границ. Время CPU — среднее по 5 проходам для изображений до 10^6 пикселей, для больших — один проход. Время GPU с копированием включает создание массива и текстурного объекта, копирование изображения на видеокарту и результата обратно (таблица 4, рисунок 2).

Таблица 4 — Сравнение CPU и GPU

Размер изображения	CPU, мс	GPU, ядро, мс	GPU, с копированием, мс	Ускорение, раз
640 × 480	3,597	0,0345	1,569	2,29
4096 × 4096	269,74	1,263	63,20	4,27
10000 × 10000	1 464,71	7,824	380,72	3,85

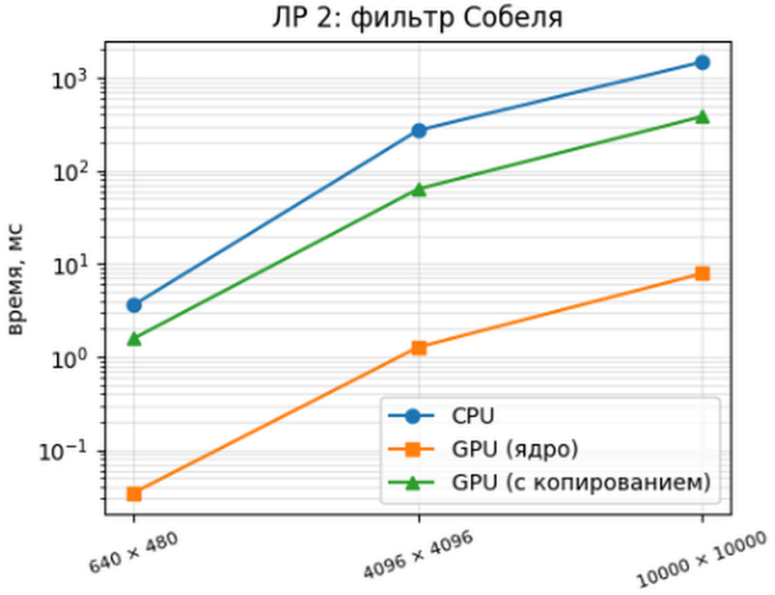


Рисунок 2 — Время фильтра Собеля на CPU и GPU

Ядро быстрее CPU в 104, 214 и 187 раз, а с учётом подготовки данных и копирования ускорение составляет 2,29, 4,27 и 3,85 раза. Для изображения 10000 × 10000 на ядро приходится только 2% полного времени GPU, остальные 98% занимают передача 400 МБ по PCIe в обе стороны и подготовка массива и текстуры. Поэтому для одиночного применения фильтра выигрыш ограничен скоростью передачи данных; он значительно больше, если изображение остаётся на видеокарте для последующих операций.

Выводы

Метод Собеля применяется для выделения контуров и границ объектов: это этап предварительной обработки в задачах компьютерного зрения (сегментация, поиск

признаков, в том числе первый шаг детектора Кэнни), обработки медицинских и спутниковых снимков. Фильтр чувствителен к шуму, поэтому перед ним часто применяют размытие.

Задача хорошо ложится на GPU: пиксели обрабатываются независимо, а обращения к соседним пикселям выигрывают от текстурного кэша. Текстурная память упростила программу: клампинг на границах выполняется аппаратно. Основные сложности: работа с API текстурных объектов, согласование координат (центр текселя, +0,5), формат изображений и допуск ±1 при округлении.

Замеры на Tesla T4 показали сильную зависимость времени от конфигурации запуска: сдаваемая конфигурация `<<<(16, 16), (32, 32)>>>` быстрее запуска одним варпом в 222 раз, а дальнейшее увеличение сетки эффекта не даёт, так как устройство уже загружено. Ядро быстрее последовательной версии на CPU в 104–214 раз, а с учётом копирования данных ускорение составляет 2,29–4,27 раза: основную долю времени занимает передача изображения между CPU и GPU.